



INDIA.RUNS

Build what next India runs on

Team Name : AI Recruit

Team Leader Name : Charishma Kottapalli

Problem Statement : Build an AI system that ranks candidates the way a great recruiter would — understanding career trajectory, skills, and behavioral fit beyond keyword matching.

Solution Overview

- What is your proposed solution?

- AI Recruit is a Hybrid Intelligence Ranking System that mirrors human recruiter reasoning.
- Stage 1 (Semantic Search): Uses Sentence-BERT vectors to evaluate conceptual similarity between job descriptions and resumes.
- Stage 2 (LLM Reasoner): Conducts deep evaluation of career growth, leadership signals, and open-source contributions.

What differentiates your approach?

- Beyond Keywords: Understands that 'FastAPI' satisfies an 'API Design' requirement without exact string matching.
- Holistic Profiling: Evaluates soft skills, mentorship history, and technical blogging alongside hard skills.
- Instant Explainability: Every score is accompanied by a transparent, AI-generated justification.

JD Understanding & Candidate Evaluation

● Key Requirements Extracted from JD:

- Structured Parsing: Automatically extracts job titles, required skill vectors, and seniority expectations.
- Semantic Mapping: Converts raw requirements into rich 384-dimensional vector representations.

Critical Candidate Evaluation Signals:

- Core Competencies (50% Weight): Direct and semantic overlap with mandatory job skills.
- Seniority & Trajectory (30% Weight): Career level alignment (e.g., matching Senior Architect roles with progressive experience).
- Platform Activity (10% Weight): Verified GitHub contributions, open-source maintenance, and active engineering presence.
- Behavioral Traits (10% Weight): Evidence of strategic planning, team leadership, and mentorship.

Ranking Methodology

● Four-Stage Ranking Pipeline:

1. Ingestion & Cleaning: Structured CSV parsing into validated Python dataclasses.
2. Vector Encoding: Sentence-BERT (all-MiniLM-L6-v2) generates high-dimensional embeddings.
3. Coarse Filtering: Cosine Similarity rapidly ranks candidates to identify the top-10 shortlist.
4. Fine-Grained Scoring: LLM multi-signal evaluation computes final normalized scores (0.0 to 1.0).

Core Algorithms & Heuristics:

- Vector Similarity: Cosine distance measures conceptual relevance across disparate terminologies.
- Weighted Multi-Signal Heuristic: Balances technical skills with behavioral and community signals.
- Zero-Dependency Fallback: Integrated TF-IDF matrix scoring ensures robust runtime reliability.

Explainability & Data Validation

● Transparent Ranking Decisions:

- Human-Readable Proof: Each candidate output includes an explicit justification string.
- Clear Attribution: Details exact skill matches, seniority alignment, and bonus signals.
- Example Output: 'Matched skills: Python, AWS; Seniority aligned; Active open-source contributor.'

Preventing Hallucinations & Bias:

- Strict Grounding: LLM evaluation is strictly bounded to verified CSV input columns.
- Deterministic Math: Base similarity is mathematically anchored by vector cosine distance.

Handling Low-Quality or Incomplete Profiles:

- Graceful Degradation: Missing optional fields (like platform activity) simply yield zero bonus without penalizing core skill scores.
- Outlier Protection: Malformed profiles are safely sanitized during data ingestion.

End-to-End Workflow

- Step-by-Step Execution Flow:

- Step 1: Input — Recruiter uploads candidate and job description CSVs via the interactive web dashboard.
- Step 2: Processing — Backend parses records and normalizes text fields for NLP encoding.
- Step 3: Embedding — S-BERT transforms profiles into 384-dimensional semantic vector space.
- Step 4: Shortlisting — Cosine similarity filters and ranks the top talent pool instantaneously.
- Step 5: AI Evaluation — LLM reasoner synthesizes hard and soft signals into final rankings.
- Step 6: Delivery — Interactive dashboard presents score rings, skill badges, and exportable CSV reports.

System Architecture

1. PRESENTATION LAYER (Web Dashboard / CLI)

↳ HTML5 Glassmorphism UI, Reactive Particle Canvas, Flask REST API (Port 5000)



2. DATA INGESTION & SANITIZATION MODULE

↳ CSV Parser, Dataclass Validation, Text Normalization & Clean-up



3. COARSE RANKING ENGINE (Semantic Vector Space)

↳ Sentence-BERT (all-MiniLM-L6-v2) Embeddings → Cosine Similarity Filtering



4. FINE-GRAINED LLM REASONER (Multi-Signal Evaluation)

↳ Skills (50%) + Seniority (30%) + Activity (10%) + Behavioral Traits (10%)



5. EXPLAINABLE DELIVERY LAYER

↳ AI Justification Generator, Visual Score Rings, Exportable ranked_candidates.csv

Results & Performance

● Demonstrated Ranking Quality:

- Accurate Prioritization: Properly ranks high-skill matches over pure years of experience when core technical alignment is superior.
- Holistic Differentiation: Effectively separates candidates with identical skill titles by evaluating open-source contributions and leadership traits.

Runtime & Compute Efficiency:

- High-Speed Execution: Full pipeline evaluates and ranks 100+ candidates in under 2 seconds.
- Lightweight Architecture: Optimized vector math runs efficiently on standard CPU hardware without requiring expensive GPUs.
- Self-Contained Reliability: Integrated fallback scoring guarantees zero downtime even in offline or air-gapped environments.

Technologies Used

● Frontend Architecture:

- HTML5 & CSS3: Premium Navy & Gold glassmorphism design with responsive grid layouts.
- Vanilla JavaScript (ES6+): Interactive particle physics canvas and 3D hover micro-animations.

Backend & API:

- Python 3.12 & Flask: Lightweight, robust REST API serving structured JSON responses.

Machine Learning & NLP Stack:

- Sentence-Transformers (S-BERT): 'all-MiniLM-L6-v2' for high-accuracy semantic embeddings.
- Scikit-Learn: Cosine similarity vector distance and TF-IDF matrix fallback.
- Pandas & NumPy: High-performance data manipulation and CSV ingestion.

Why This Stack? Maximum speed, zero frontend build complexity, and proven NLP accuracy.

Submission Assets

Official GitHub Repository:

- <https://github.com/RUSHI-KOLLA/ai-recruitment> (Clean, documented, and ready to run)

Included Submission Deliverables:

- Complete codebase with web dashboard and CLI pipeline
- AI_Recruit_Submission.pdf (Executive approach presentation)
- ranked_candidates.csv (Verified output formatting)

Team Members (AI Recruit):

- Charishma Kottapalli (Leader) — kottapallicharishma@gmail.com
- Kolla Rushikesh — kollarushi2006@gmail.com
- Sahil Kumar — kumarsahil282006@gmail.com
- Gaurav Singh — g.singh200293@gmail.com



INDIA.RUNS

Build what next India runs on

THANK YOU